Stock Exchange Simulator - Architecture & Service Design

1. Project Goal

Build a Zerodha-like stock exchange simulator using Java, Spring Boot, Kafka, Redis, WebSockets and Microservices. The system should support live stock prices, order placement, order matching, portfolio management, wallet management, and event-driven trade execution.

2. High-Level Architecture

Frontend (React) -> API Gateway -> User Service, Wallet Service, Order Service, Portfolio Service, Stock Service. Kafka acts as the event backbone. Matching Engine consumes orders and publishes trades. Redis stores frequently accessed data. MySQL/PostgreSQL stores transactional data. WebSocket broadcasts live stock prices.

3. User Service

Responsibilities: - Registration/Login - JWT Authentication - User Profile Management Tables: users(id, username, email, password_hash, created_at) APIs: POST /register POST /login GET /profile PUT /profile

4. Wallet Service

Responsibilities: - Deposit Funds - Withdraw Funds - Block Funds during order placement - Release Funds on cancellation - Transfer Funds after trade execution Tables: wallet(user_id, balance, blocked_balance) Events Consumed: TradeExecutedEvent Events Produced: WalletUpdatedEvent

5. Stock Service

Responsibilities: - Maintain stock master data - Current stock prices - Historical prices - Publish price updates Tables: stocks(symbol, current_price) stock_price_history(id, symbol, price, timestamp) Redis Cache: stock:TCS stock:INFY stock:RELIANCE

6. Order Service

Responsibilities: - Place Buy/Sell Orders - Cancel Orders - Query Order Status Tables: orders(id, user_id, symbol, type, quantity, price, status) Statuses: OPEN MATCHED PARTIALLY_MATCHED CANCELLED Produces: OrderPlacedEvent OrderCancelledEvent

7. Matching Engine

Core Component. Data Structures: PriorityQueue PriorityQueue Rules: - Highest buy price gets priority - Lowest sell price gets priority - Match when buy_price >= sell_price Produces: TradeExecutedEvent

8. Portfolio Service

Responsibilities: - Maintain user holdings - Calculate average buy price - Calculate portfolio value Table: portfolio(user_id, symbol, quantity, average_price) Consumes: TradeExecutedEvent

9. Notification Service

Responsibilities: - Trade notifications - Order completion notifications - Email/SMS/Push integration (future) Consumes: TradeExecutedEvent WalletUpdatedEvent

10. Kafka Topics

order-events trade-events wallet-events price-events notification-events Event Flow: Order -> Matching Engine -> Trade -> Wallet/Portfolio -> Notification

11. WebSocket Design

Endpoint: /ws/stocks Purpose: Broadcast live stock price updates to all connected clients. Frontend subscribes using STOMP/WebSocket.

12. Saga Pattern

Trade Executed -> Wallet Updated -> Portfolio Updated -> Notification Sent If Portfolio Update Fails: -> Compensation Transaction -> Refund Wallet -> Mark Trade Failed

13. Redis Usage

Cache: - Latest stock prices - Portfolio summaries - Top gainers/losers Benefits: - Faster reads - Reduced database load

14. Database Schema Summary

users wallet orders trades portfolio stocks stock_price_history

15. Development Roadmap

Phase 1: Monolith with User, Wallet, Order, Portfolio, Stock modules. Phase 2: Introduce Kafka and Matching Engine. Phase 3: Add WebSocket live updates. Phase 4: Split into Microservices. Phase 5: Add Redis, Saga Pattern, Idempotency, DLQ and Docker/Kubernetes.